Integración en MATLAB y Sistemas Lineales

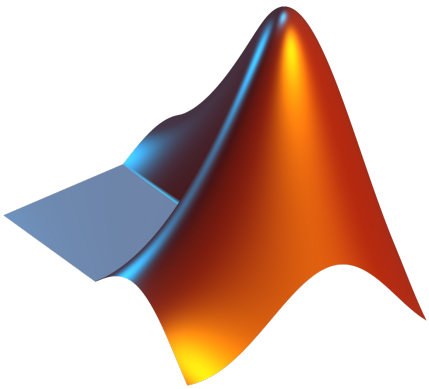
Fabrizio Benavidez[1]

https://github.com/Fabopy/Matlab-curso-

[1] *Facultad de Ciencias Exactas, Físicas y Naturales, Ingeniería Aeronáutica*

Resumen En el siguiente trabajo se realiza integración numérica en MATLAB mediante algunos métodos propuestos. Se utiliza la regla del Trapecio, Simpson y además la función propia de MATLAB para integración. Además, se plantea el problema de resolver la integración como ecuación diferencial de primer grado, utilizando funciones de Runge-Kutta para su solución.
Por otro lado, se estudia el caso de un brazo robótico para estimar su función de transferencia, proponiendo un sistema de segundo orden.

Palabras claves: Integración, MATLAB, Modelo



1 de noviembre de 2026

Semana 11

[1] fabrizio.benavidez@mi.unc.edu.ar

Guía 21: Integración en MATLAB

Veamos la siguiente función

$$f(t) = e^{\cos(-1/t^2)} \tag{1}$$

El objetivo es encontrar una primitiva para evaluarla entre 0,1 y 3, esto es:

$$\int_{0,1}^{3} f(t)dt \tag{2}$$

La siguiente figura representa el gráfico de la función

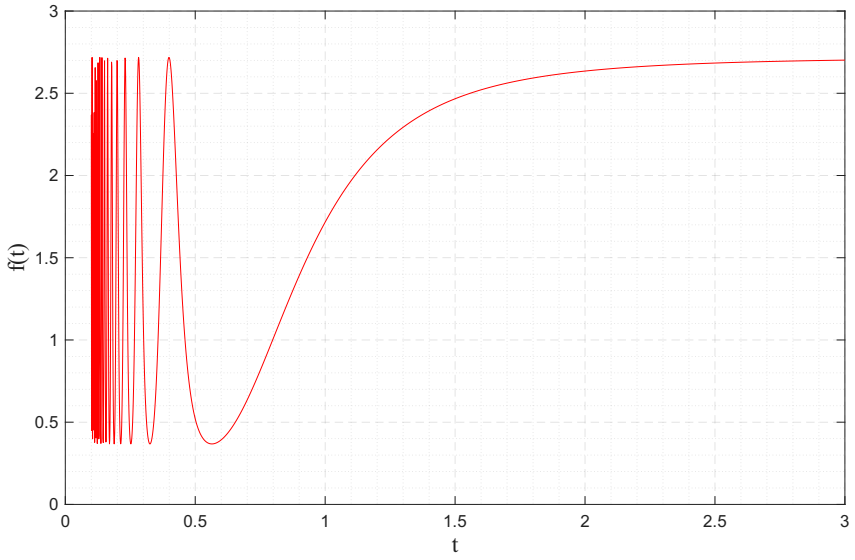


Figura 1. Gráfico de $f(t)$

Se observa que integrar es mucho más fácil entre $[0{,}5, 3]$ pero en $[0{,}1, 0{,}5)$ la integración es mucho más difícil debido a las oscilaciones en esa zona, por lo tanto se requiere un paso de integración más chico para considerar los cambios bruscos cercanos al 0.

Claramente la función es no elemental y por lo tanto no se puede encontrar una primitiva. La única forma de evaluar la integral es mediante método numéricos y distintos métodos que se pueden aplicar en MATLAB. En la figura 2 se observa la función $g(t)$ y los polinomios de interpolación de primer grado (trapecio) e interpolación de segundo grado (simpson). Por lo tanto, podemos aplicar dichos métodos para aproximar la función. La fórmula del trapecio es:

$$\int_a^b f(t) \approx \frac{h}{2}\left(f(t_0) + 2\left(\sum_{i=1}^{n-1} f(t_i) + \right) f(t_n)\right) \tag{3}$$

La fórmula de Simpson

$$\int_a^b f(t)dt \approx \frac{h}{3}\left[f(t_0) + 4\left(\sum_{i=1, i \text{ impar}}^{n-1} f(t_i)\right) + 2\left(\sum_{i=2, i \text{ par}}^{n-2} f(t_i)\right) + f(t_n)\right] \tag{4}$$

El siguiente código hecho en MATLAB calcula, de forma numérica y con la propia función `integral` de MATLAB, la integral de (2) con un paso de 0,05.

```
clc
close all

h = 0.05; %paso
```

```
t = 0.1:h:3;
ft = exp(cos(-1./t.^2));
%{
Las partes mas faciles para integrar esta funcion
es desde 0.5 hacia adelante. Aunque cumple con
las condiciones de integrabilidad (segun las condiciones
de Riemann), numericamente es dificil resolverlo.
Las partes problematicas son las oscilaciones que tiene
al principio
%}

plot(t, ft, 'r')


%Podemos usar el metodo Simpson o del trapecio

function S = trapecio(func, h)
    %{
    func : funcion a encontrar la integral
    h : paso de integracion
    %}
    S = h/2*(func(1) + 2*sum(func(2:end-1)) + func(end));
end


function S = simpson(func, h)
    A = h/3*(func(1) + func(end));
    n = length(func);
    for i = 2:1:n-1
        if mod(i, 2) == 0
            A = A + h/3*4*func(i);
        else
            A = A + h/3*2*func(i);
        end
    end
    S = A;
end


ft_trapecio = trapecio(ft, h);
>>  6.14458158849869

ft_simpson = simpson(ft, h);
>>  6.09083232859325

ft_matlab = integral(@(t) exp(cos(-1./t.^2)), 0.1, 3);
>> 6.03513177870255
```

Si ahora hacemos que el paso sea 0,001 se obtienen resultados muy próximos al valor de 6,035.

Podemos resolver la integral mediante métodos de ecuaciones diferenciales. Consideremos la expresión de la integral de $f(t)$, es decir:

$$F(t) = \int f(t)dt \tag{5}$$

Por el teorema fundamental del cálculo

$$\dot{F}(t) = f(t) \tag{6}$$

donde el lado derecho no es más que la expresión (1). Tenemos una ecuación diferencial de primer grado y por lo tanto podemos aplicar métodos como Runge-Kutta de cuarto orden.

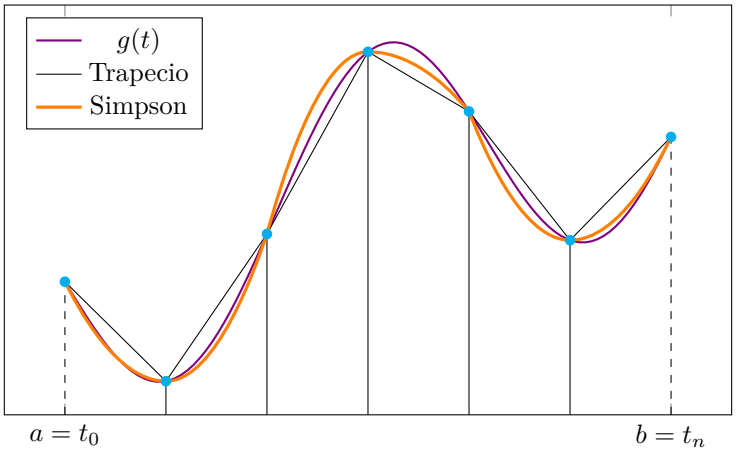


Figura 2. Aproximación de una integral mediante la regla del trapecio y la regla de Simpson.

El siguiente código en MATLAB representa la solución a la ecuación diferencial

```
%Mediante una ecuacion diferencial

function [df] = f21(t, y)
    df = exp(cos(-1./t.^2));
end

opts = odeset('RelTol',1e-7,'AbsTol',1e-4);
%Si agrego el opts mejora la precision
%https://la.mathworks.com/help/matlab/ref/ode45.html?s_tid=
    ta_help_results
%Primer Metodo
[T1, Y1] = ode45(@f21, [0.1, 3], 0.1, opts);
ft_ode45 = Y1(end) - Y1(1);
>> 6.03679802230507

%Segundo metodo
[T2, Y2] = ode15s(@f21, [0.1, 3], 0.1, opts);
ft_ode15 = Y2(end) - Y2(1);
>> 6.03098868384105

%Tercer metodo
[T3, Y3] = ode23s(@f21, [0.1, 3], 0.1, opts);
ft_ode23 = Y3(end) - Y3(1);
>> 6.03404436431533
```

La función `ode45`, `ode15s`, `ode23s` reciben, principalmente, 3 argumentos: el lado derecho a resolver, el `tspan` o dominio de integración y las condiciones iniciales. El cuarto argumento sirve para agregar opciones de tolerancias a errores absolutos y relativos. Aunque las 3 funciones sirven para integrar ecuaciones diferenciales se diferencian en:

- `ode45` : Utiliza Runge-Kutta de cuarto orden. Tiene propósitos generales pero no funciona muy bien para ecuaciones diferenciales rígidas (cambios muy bruscos).
- `ode23s` : Utiliza Runge-Kutta de segundo orden. Aunque sea menos preciso que RK4(5) sirve para resolver ecuaciones diferenciales rígidas.
- `ode15s` : está basado en fórmulas de diferenciación numérica.

Se observa que cambiando la tolerancia relativa ésta misma cambia los valores de los resultados. Sí se puede confiar en los resultados dados por los métodos numéricos, porque

a pesar de que son aproximaciones los resultados son certeros debido a que también existe un resultado de los errores para cada iteración. Si sabemos cómo controlar estos errores (mejorando el método, cambiando las tolerancias, reduciendo el paso, etcétera) entonces podemos asegurar los resultados. Cabe aclarar que juega mucho el conocimiento sobre el tema y saber interpretar los resultados, porque pese a que MATLAB pueda resolver ecuaciones diferenciales, los resultados tienen que tener sentido y eso dependerá del análisis y fenómeno el cual se trata.

Por último, definamos variables simbólicas para integrar en MATLAB. Veamos lo siguiente:

```
syms x z a
f = 3*a+x;
int_f = int(f); %Integra con respecto a x si no pongo nada
>> (x*(6*a + x))/2
```

Integró con respecto a la variable `x`. Ahora, si especificamos que queremos integrar con respecto a la variable `a`, entonces

```
syms x z a
f = 3*a+x;
int_f = int(f, a);
>> (a*(3*a + 2*x))/2
```

Veamos ahora el ejemplo de integrar $g(x, z) = x/(1 + z^2)$ con respecto a la variable x y z.

```
g = x / (1+z^2);
int_g_x = int(g, x);
>> x^2/(2*(z^2 + 1))

int_g_z = int(g, z);
>> x*atan(z)
```

Donde `atan` representa el arcotangente.

Por último, veamos el ejemplo de integrar la siguiente función

$$f(x) = e^{x^2} \tag{7}$$

con respecto a x. Si hacemos esto en MATLAB obtenemos lo siguiente:

```
h = exp(x^2);
int_h = int(h);
>> (pi^(1/2)*erfi(x))/2
```

Para saber qué función es erfi, nos basamos en el libro *Table of Integrals, Series, and Products* de I.S. Gradshteyn y I.M. Ryzhik (página 157, sección 2.33, fórmula 2):

$$\int e^{ax^2} dx = \frac{1}{2}\sqrt{\frac{\pi}{a}}\text{erfi}\,(\sqrt{a}x) \tag{8}$$

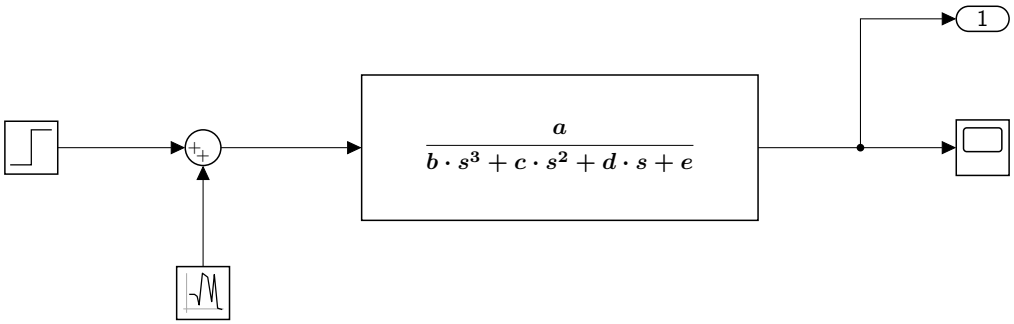
donde para nuestro caso $a = 1$ y $\text{erfi} = \text{erf}(ix)/i$, donde

$$\text{erf}(ix) = \frac{2}{\sqrt{\pi}}\int_0^{ix} e^{-t^2} dt \tag{9}$$

es la función error.

Guía 22: Identificadores de Sistemas Lineales

Implementemos el siguiente sistema en Simulink:



donde

$$
\begin{aligned}
a &= 1000 + 1100 \text{ rand} \\
b &= \text{round(rand)} \\
c &= 40 + 5 \text{ rand} \\
d &= 100 + 50 \text{ rand} \\
e &= 1000 + 200 \text{ rand}
\end{aligned}
$$

son los parámetros de la función transferencia.

La siguiente figura representa la salida para el sistema



Figura 3. Respuesta del brazo robótico ante una entrada escalón

Dado que no conocemos nuestro sistema podemos suponer que es un sistema de segundo orden:

$$G(s) = k \, \frac{\omega^2}{s^2 + 2\zeta\omega + \omega^2} \tag{10}$$

donde

$$k = \frac{\sum_{i=n-49}^{n} y_i}{50} \tag{11}$$

$$\zeta = \sqrt{\frac{-2\log\left(\frac{y_{\text{máx}}}{k} - 1\right)}{\pi^2 - 2\log\left(\frac{y_{\text{máx}}}{k} - 1\right)}} \tag{12}$$

$$\omega = \frac{1{,}1 + 0{,}125\zeta + 0{,}469\zeta^2}{t_d} \tag{13}$$

El tiempo de retardo, t_d, es el tiempo que tarda el sistema para llegar a la mitad de su valor final. El siguiente código de MATLAB muestra cómo calcular este valor, implementado el sistema $G(s)$:

```
%Disminuimos el ruido tomando los ultimos 50 valores
n = length(y);
k = sum(y(n - 49:end)/50);

function zeta = factor_amortiguamiento(y, k)
    ymax = max(y);
    num = -2*log(ymax/k - 1);
    den = pi^2 - 2*log(ymax/k - 1);
    zeta = sqrt(num/den);
end

%Factor de amortiguamiento
zeta = factor_amortiguamiento(y, k);
indice = find(y > 0.5*k, 1, 'first');
%Calculo del tiempo de retardo
td = t(indice);
%frecuencia natural del sistema
omega = (1.1 + 0.125*zeta + 0.469*zeta^2)/td;

%Armamos la funciin transferencia de segundo orden
G = tf([k*omega^2], [1, 2*omega*zeta, omega^2]);

[yout, tout] = step(G);


subplot(2,1,1)
plot(t, y, 'r')
hold on
step(G, 'm')
subplot(2,1,2)
plot(t(1:400), y(1:400), 'r')
hold on
plot(tout, yout, 'm')
```

La figura 4 representa la respuesta del modelo ante el sistema real. Por último, veamos el sistema cuando éste no presenta ruido, esto es `sis_no_ruido = tf([a], [b, c, d, e]);`. En la figura 5 se observa distintos diagramas que representan a los dos sistemas. Se observa que los polos de ambos son prácticamente iguales.

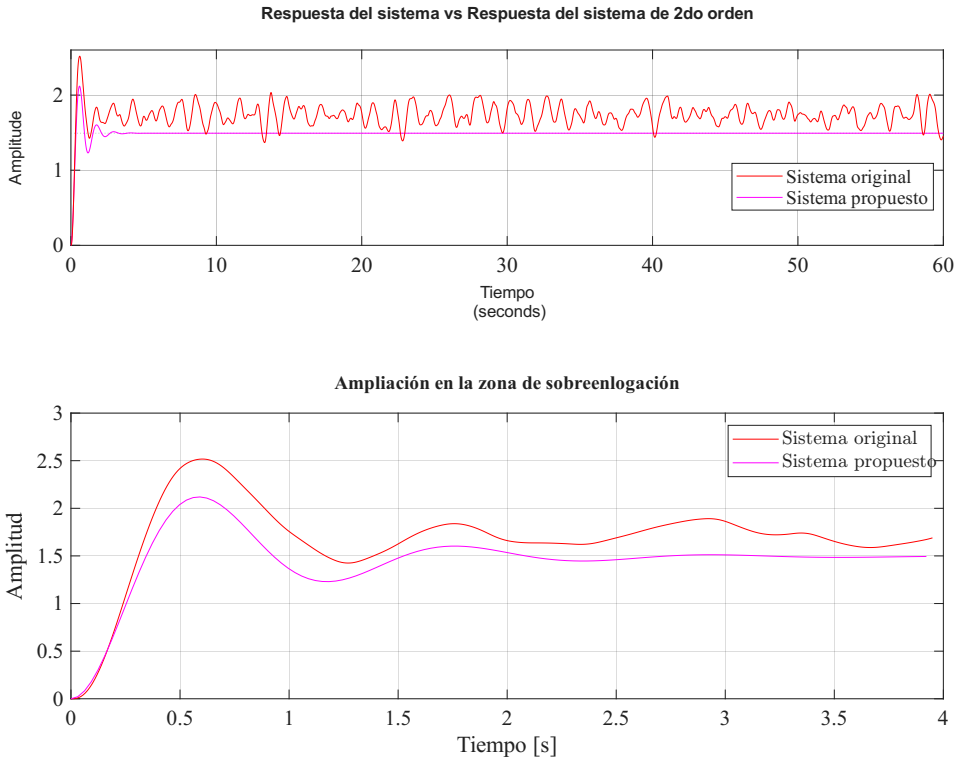


Figura 4. Respuestas del modelo propuesto vs el sistema que representa el movimiento del robot.

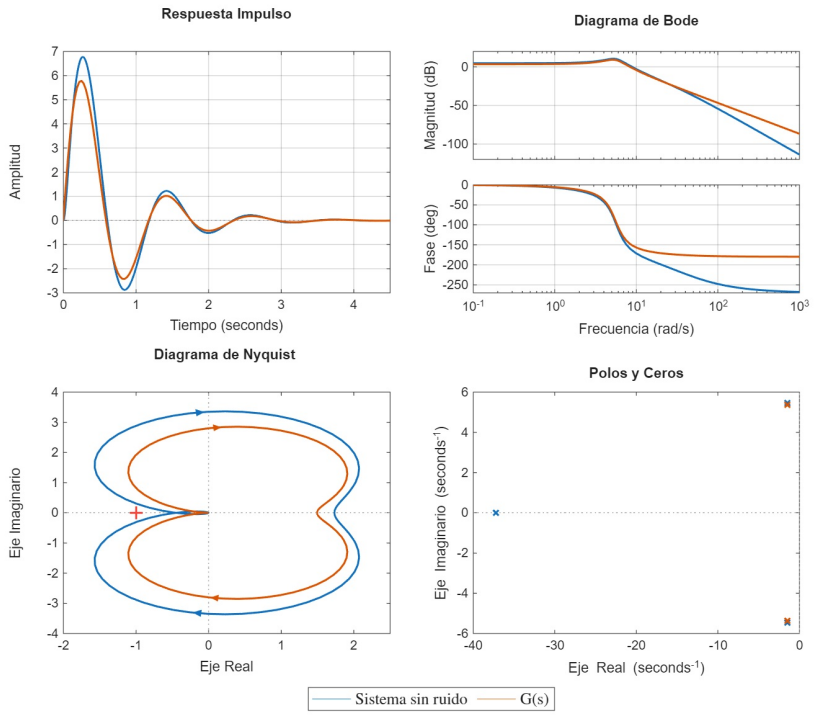


Figura 5. Comparación del sistema sin ruido con respecto al sistema propuesto de segundo orden.

En la figura 6 se compara algunos tiempos ante la entrada escalón

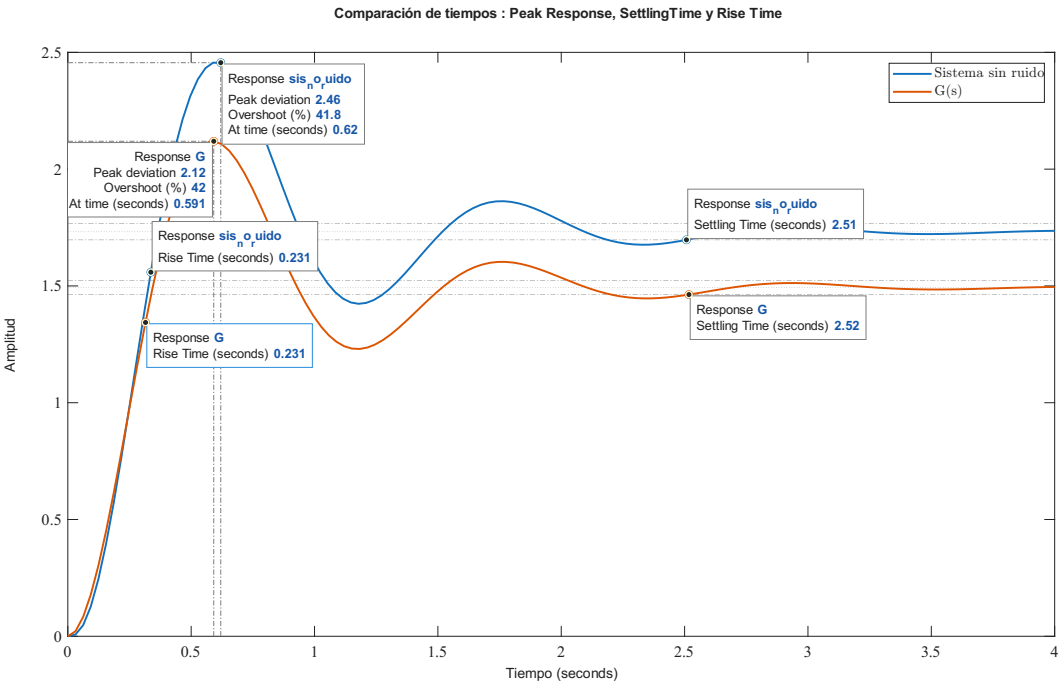


Figura 6. Algunas características de los sistemas ante la entrada escalón.

Por último, se mencionan algunos errores que se cometen al momento de estimar la función de transferencia $G(s)$:

1. El orden del sistema o modelo.
2. Utilizar datos pocos fiables.
3. Estimar mal sus parámetros.
4. Validarlo de manera inadecuada: concluir que el modelo representa una buena estimicación al sistema original.